

Documento de Diseño — API REST Gimnasio

Proyecto: API REST con persistencia JPA — UT6

Autores: Jonay Martín Rivero González · Jeremi González

Módulo: Programación (PRO) — 1ºB DAM · Curso 2025-2026

Fecha de entrega: 5 de junio de 2026

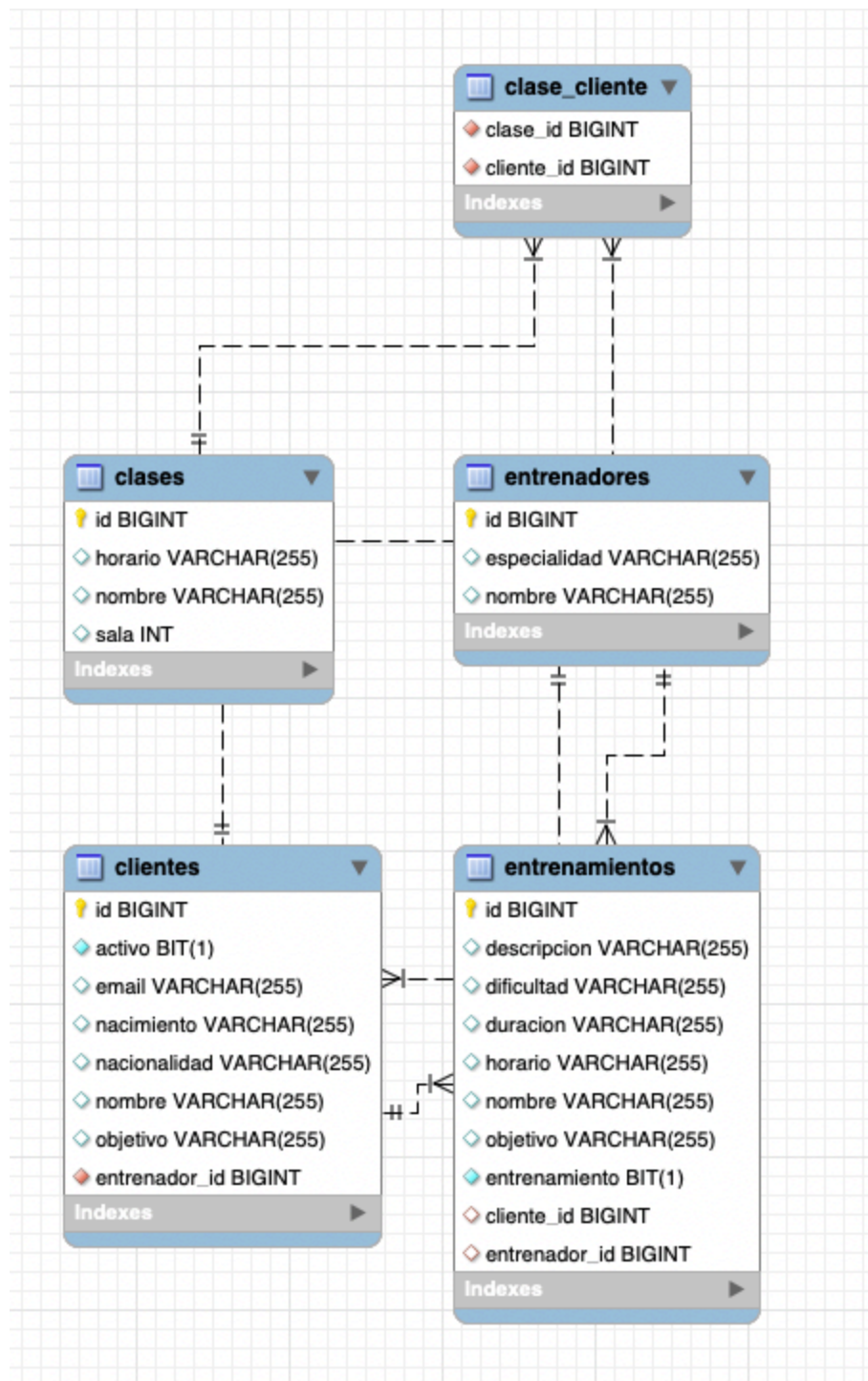
1. Descripción del dominio

El proyecto gestiona un gimnasio. Las entidades principales son:

- **Entrenador:** profesional que dirige clientes y entrenamientos.
- **Cliente:** persona inscrita en el gimnasio, asignada a un entrenador.
- **Clase:** actividad grupal a la que se pueden apuntar varios clientes.
- **Entrenamiento:** sesión individual o grupal con datos de dificultad, duración y horario.

Se eligió este dominio porque permite representar de forma natural los tres tipos de relación JPA requeridos por la actividad: @ManyToOne, @OneToMany y @ManyToMany.

2. Diagrama Entidad-Relación



Relaciones:

Relación	Tipo	Descripción
Cliente → Entrenador	@ManyToOne	Cada cliente tiene un entrenador asignado. FK entrenador_id en tabla clientes.
Entrenador → Clientes	@OneToMany	Inverso de la anterior. Mapeado con mappedBy.
Clase ↔ Cliente	@ManyToMany	Un cliente puede estar en varias clases y una clase tiene varios clientes. Tabla intermedia clase_cliente.
Entrenamiento → Cliente	@ManyToOne	Cada entrenamiento puede estar asociado a un cliente.
Entrenamiento → Entrenador	@ManyToOne	Cada entrenamiento puede estar asociado a un entrenador.

3. Tablas generadas en la base de datos

Al arrancar la aplicación con `spring.jpa.hibernate.ddl-auto=update`, Hibernate genera automáticamente las siguientes tablas en la base de datos `gimnasiodb`:

Tabla entrenadores

Columna	Tipo	Restricción
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR(255)	NOT NULL
especialidad	VARCHAR(255)	NOT NULL

Tabla clientes

Columna	Tipo	Restricción
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR(255)	NOT NULL
nacionalidad	VARCHAR(255)	
nacimiento	VARCHAR(255)	NOT NULL
objetivo	VARCHAR(255)	
email	VARCHAR(255)	NOT NULL
activo	TINYINT(1)	NOT NULL
entrenador_id	BIGINT	FK → entrenadores(id)

Tabla clases

Columna	Tipo	Restricción
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR(255)	
horario	VARCHAR(255)	
sala	INT	

Tabla clase_cliente (*tabla intermedia @ManyToMany*)

Columna	Tipo	Restricción
clase_id	BIGINT	FK → clases(id)
cliente_id	BIGINT	FK → clientes(id)

Tabla entrenamientos

Columna	Tipo	Restricción
id	BIGINT	PK, AUTO_INCREMENT
nombre	VARCHAR(255)	
descripcion	VARCHAR(255)	
objetivo	VARCHAR(255)	
dificultad	VARCHAR(255)	
duracion	VARCHAR(255)	
horario	VARCHAR(255)	
entrenamiento	TINYINT(1)	
cliente_id	BIGINT	FK → clientes(id)
entrenador_id	BIGINT	FK → entrenadores(id)

Nota: Las capturas de las tablas generadas en MySQL Workbench / DBeaver se adjuntan en la defensa oral en directo.

4. Lista completa de endpoints

Clientes

Método	Ruta	Descripción	Auth	Respuesta
GET	/api/v1/clientes	Lista todos los clientes	No	200 OK
GET	/api/v1/clientes?nombre=xxx	Busca por nombre (contiene, ignora mayúsculas)	No	200 OK
GET	/api/v1/clientes/{id}	Obtiene cliente por ID	No	200 OK / 404

Método	Ruta	Descripción	Auth	Respuesta
GET	/api/v1/clientes/entrenador/{id}	Lista clientes de un entrenador concreto	No	200 OK
POST	/api/v1/clientes	Crea un nuevo cliente	Sí	201 Created
PUT	/api/v1/clientes/{id}	Actualiza un cliente	Sí	200 OK / 404
DELETE	/api/v1/clientes/{id}	Elimina un cliente	Sí	204 No Content / 404

Entrenadores

Método	Ruta	Descripción	Auth	Respuesta
GET	/api/v1/entrenadores	Lista todos los entrenadores	No	200 OK
GET	/api/v1/entrenadores/{id}	Obtiene entrenador por ID	No	200 OK / 404
POST	/api/v1/entrenadores	Crea un nuevo entrenador	Sí	200 OK
PUT	/api/v1/entrenadores/{id}	Actualiza un entrenador	Sí	200 OK / 404
DELETE	/api/v1/entrenadores/{id}	Elimina un entrenador	Sí	204 No Content

Clases

Método	Ruta	Descripción	Auth	Respuesta
GET	/api/v1/clases	Lista todas las clases	No	200 OK
GET	/api/v1/clases/{id}/aforo	Devuelve el nº de clientes inscritos (JPQL @Query)	No	200 OK

Método	Ruta	Descripción	Auth	Respuesta
POST	/api/v1/clases	Crea una nueva clase	Sí	201 Created
POST	/api/v1/clases/{claseId}/clientes/{clienteId}	Inscribe un cliente en una clase	Sí	200 OK / 404
DELETE	/api/v1/clases/{id}	Elimina una clase	Sí	204 No Content / 404

Entrenamientos

Método	Ruta	Descripción	Auth	Respuesta
GET	/api/v1/entrenamientos	Lista todos los entrenamientos	No	200 OK
GET	/api/v1/entrenamientos/{id}	Obtiene entrenamiento por ID	No	200 OK / 404
POST	/api/v1/entrenamientos	Crea un nuevo entrenamiento	Sí	200 OK
PUT	/api/v1/entrenamientos/{id}	Actualiza un entrenamiento	Sí	200 OK / 404
DELETE	/api/v1/entrenamientos/{id}	Elimina un entrenamiento	Sí	204 No Content

5. Decisiones técnicas justificadas

Base de datos: MySQL en lugar de H2

Se eligió **MySQL** en lugar de H2 porque permite visualizar las tablas de forma persistente con herramientas como MySQL Workbench, lo que facilita la demostración en la defensa. Con H2 en memoria los datos se pierden al reiniciar y la consola embebida es menos visual. La configuración está completamente documentada en `application.properties`.

Uso de Lombok

Se usó `@Data`, `@NoArgsConstructor` y `@AllArgsConstructor` para eliminar código boilerplate (getters, setters, constructores, equals, hashCode, toString).

`@JsonIgnore` y `@JsonIgnoreProperties` - por qué y dónde

En la relación Entrenador: clientes se usa `@JsonIgnore` porque si se serializa un Entrenador con su lista de Cliente y cada Cliente contiene su Entrenador, se produce un bucle infinito de JSON. El `@JsonIgnore` se coloca en el lado `@OneToMany` (la lista en Entrenador) porque es el lado que genera la recursión al serializar. También se usa `@ToString.Exclude` por la misma razón en el método `toString()` generado por Lombok.

En la relación `@ManyToMany` entre Clase y Cliente se usa `@JsonIgnoreProperties("clases")` en lugar de `@JsonIgnore` completo, porque así se evita el bucle pero se mantiene visible la información del otro lado.

Optional - por qué nunca se llama a `.get()` directamente

Todos los `findById` devuelven `Optional<T>`. En los controllers se usa siempre `.map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build())`, que encadena el resultado sin extraer el valor directamente. Esto evita `NoSuchElementException` y expresa de forma declarativa el comportamiento ante ausencia de datos.

`@Query JPQL` - por qué no un método derivado

El método `contarClientesInscritos` en `ClaseRepository` no puede expresarse como método derivado porque requiere navegar la relación `@ManyToMany` y aplicar un `COUNT` sobre la colección asociada (`cl.clientes`). Un método derivado solo puede filtrar por campos directos de la entidad. La diferencia con SQL es que JPQL opera sobre entidades Java y sus atributos, no sobre nombres de tablas ni columnas reales.

Spring Security - HTTP Basic con GETs públicos

Se eligió HTTP Basic por su sencillez y porque el enunciado acepta cualquier alternativa justificada. La configuración permite acceso público a todos los GET (para facilitar la demo y la interfaz web) y requiere autenticación para cualquier otra operación. El usuario y contraseña se configuran en `application.properties`.

Arquitectura Controller > Service > Repository

Se respeta estrictamente la separación de responsabilidades. Los controllers solo manejan HTTP (reciben requests, devuelven `ResponseEntity`). Los services contienen la lógica de negocio (actualización campo a campo, lógica de inscripción sin duplicados en clases). Los repositories solo declaran métodos de acceso a datos extendiendo `JpaRepository`.

Validación con `@Valid` y `@ControllerAdvice`

Las entidades Cliente y Entrenador tienen `@NotBlank` y `@NotNull` en los campos obligatorios. Cuando se recibe un body inválido en un POST/PUT, Spring lanza `MethodArgumentNotValidException`, que es capturada por `GlobalExceptionHandler` y devuelta como JSON estructurado con los campos que fallaron, en lugar del error por defecto de Spring.

Interfaz web estática

Se incluye `index.html` en `src/main/resources/static/` para consumir la API con `fetch`. Spring Boot sirve automáticamente el contenido de esa carpeta. No requiere ninguna configuración adicional.

6. Estructura de paquetes

`com.dam.userapp`

- |— `controller/` # Capa HTTP — `@RestController`
- |— `service/` # Lógica de negocio — `@Service`
- |— `repository/` # Acceso a datos — `JpaRepository`
- |— `model/` # Entidades JPA — `@Entity`
- |— `config/` # Configuración de seguridad
- |— `Exception/` # Manejo global de errores — `@ControllerAdvice`

7. Cómo ejecutar y verificar

Desde la carpeta Gimnasio/

`./mvnw spring-boot:run`

Verificar que las tablas se han creado en MySQL:

USE `gimnasiodb`;

```
SHOW TABLES;
```

Las queries SQL generadas por Hibernate aparecen en la consola del servidor gracias a `spring.jpa.show-sql=true`.